

UNIVERSIDAD POLITÉCNICA SALESIANA

Ingeniería en Ciencias de la Computación

TALLER

Planificación y responsabilidades del equipo: Chat multiusuario en red LAN

Asignatura:	Sistemas Distribuidos
Docente:	Julio Proaño
Nivel / paralelo:	6to semestre / 1
Integrantes:	Ian Mena (líder) Juan Nuñez Francisco Urgiles
Fecha de entrega:	22/10/26
Período académico:	69
Sede:	Quito

Índice general

1. Introducción 1
2. Líder del equipo 1
3. Hoja de ruta 1
4. Responsabilidades y entregables 2
5. Estructura del repositorio 2
6. Ramas de trabajo 3
7. Acuerdos de trabajo 3

Índice de tablas

Tabla 1. Hoja de ruta del taller 1
Tabla 2. Responsabilidades y entregables por integrante 2
Tabla 3. Ramas de trabajo del repositorio 3

1. Introducción

Este documento recoge la planificación del taller **Chat multiusuario en red LAN por consola** y la división del trabajo del equipo. Se definió el 16/10/26, en persona, durante la clase: allí se eligió al líder, se repartieron los componentes del sistema y se acordaron los días de reunión.

El trabajo se dividió por componente: el líder define el contrato común (protocolo) e integra, Juan Nuñez construye el servidor y Francisco Urgiles el cliente. El repositorio del proyecto es `github.com/iansmena/SD_SocketsMultihilo`.

2. Líder del equipo

El equipo eligió como líder a **Ian Mena**, con las siguientes funciones:

- Coordinar al equipo y definir los días de reunión.
- Administrar el repositorio de GitHub: estructura de carpetas, ramas e integración del trabajo.
- Definir el protocolo de mensajes y las clases abstractas compartidas (`protocolo.py`).
- Revisar el código de cada componente antes de integrarlo y dirigir las pruebas en red LAN.
- Consolidar la documentación: README, acta de actividades y bitácora del líder.

3. Hoja de ruta

La práctica avanza en siete pasos. El contrato común se define antes de programar en paralelo, para que el servidor y el cliente se desarrollen sin depender uno del otro, y la documentación del trabajo en equipo se cierra al final.

Tabla 1. Hoja de ruta del taller

N.º	Qué se hace	Responsable	Entregable
1	Definir roles y división del trabajo	Ian	Documento de responsabilidades
2	Crear el repositorio, la estructura de carpetas y las ramas	Ian (Juan y Francisco crean su rama)	Historial en GitHub
3	Definir el contrato común: protocolo de mensajes (delimitador \n) y clases abstractas	Ian	chat/protocolo.py
4	Desarrollar en paralelo el servidor y el cliente	Juan y Francisco	servidor_chat.py y cliente_chat.py
5	Integrar y probar en red LAN (servidor en 0.0.0.0)	Todos	Capturas de la conversación
6	Documentar: ejecución paso a paso, SOLID y concurrencia	Ian (con los aportes de Juan y Francisco)	README.md
7	Cerrar el trabajo en equipo	Ian	Acta de actividades y bitácora del líder

Fuente: Elaboración propia.

4. Responsabilidades y entregables

Cada integrante es responsable de un componente y de su documentación. La Tabla 2 resume el rol, las responsabilidades y los entregables de cada uno.

Tabla 2. Responsabilidades y entregables por integrante

Integrante	Rol	Responsabilidades	Entregables
Ian Mena	Líder e integrador	Coordinar al equipo; configurar el repositorio y las ramas; definir el protocolo de mensajes y las clases abstractas compartidas; revisar e integrar el trabajo; dirigir las pruebas en LAN	protocolo.py; README integrado; documento de responsabilidades; acta; bitácora del líder; historial de GitHub
Juan Nuñez	Desarrollador del servidor	Registro dinámico de clientes conectados; difusión (broadcast) de mensajes; notificación de conexiones y desconexiones con IP y puerto; escucha en 0.0.0.0; manejo de desconexiones abruptas sin caída del servidor	servidor_chat.py documentado; aporte del servidor para el README
Francisco Urgiles	Desarrollador del cliente	Solicitud de alias al conectarse; hilo de recepción independiente del de escritura; visualización de mensajes sin interferir con la escritura; cierre limpio con Ctrl+C o pérdida de conexión	cliente_chat.py documentado; aporte del cliente para el README

Fuente: Elaboración propia.

Las pruebas en red LAN y las capturas de la conversación se realizan entre los tres, cada uno desde su propia computadora.

5. Estructura del repositorio

El código base del docente se conserva intacto como referencia del antes y el después. `protocolo.py` vive junto al servidor y al cliente para que ambos lo importen sin configuración adicional.

```
SD_SocketsMultihilo/
├── README.md
├── .gitignore
├── chat/
│   ├── protocolo.py          # Ian: contrato común
│   ├── servidor_chat.py      # Juan
│   └── cliente.py       # Francisco
├── codigo_base/             # serverMH.py y clientMH.py originales
└── docs/
    ├── 01_responsabilidades.pdf
    ├── 02_acta.pdf
    ├── 03_bitacora_lider.pdf
    └── evidencias/           # server.png y cliente.png
```

6. Ramas de trabajo

Nadie sube cambios directamente a `main`: cada integrante trabaja en su rama y la integra mediante un Pull Request revisado por el líder.

Tabla 3. Ramas de trabajo del repositorio

Rama	Responsable	Contenido
main	Ian Mena	Versión estable e integrada (solo recibe Pull Requests)
feature/protocolo	Ian Mena	Protocolo de mensajes y clases abstractas
feature/servidor	Juan Nuñez	Servidor de difusión
feature/cliente	Francisco Urgiles	Cliente concurrente
docs/readme	Ian Mena	README integrado y documentación

Fuente: Elaboración propia.

7. Acuerdos de trabajo

- Cada integrante desarrolla y prueba su componente en su propia computadora.
- Las reuniones se realizan por Discord; el grupo de WhatsApp se usa para acordar los horarios.
- Las dudas y los problemas se comunican en las reuniones y se resuelven entre todos.
- No se fijan fechas límite por tarea: el avance se revisa en cada reunión acordada.